REPORT (Part II)

Test instances

In Part I we picked three of Pisinger's low-dimensional instances [Pis] (f4, f7, f1) sized by number of items: 4, 7 and 10. That is the natural notion of size for a knapsack, but it is not what limits us here. When we started designing T2 we realised that the circuit needs more than one qubit per item: the oracle has to compute the total weight and the total value inside the circuit, in two's complement registers, so that it can read the sign bit of each constraint `e <= 0`. The size of those registers depends on the numbers, not on the number of items.

We therefore estimate the circuit size as:

```
qubits = n (one per item) + bits(Σw·y − C − 1) + bits(V −
```

where `bits(e)` is the smallest two's complement width that holds every value `e` can take (we store `e − 1` because the sign bit tells us `e − 1 < 0`, which for integers is the same as `e <= 0`). For f4, for example, `Σw·y − 11 − 1` ranges from –12 to 7 (5 bits) and `23 − Σv·y − 1` from –19 to 22 (6 bits), giving 4 + 5 + 6 = 15 qubits. This is a lower bound: depending on the adder used in T2, a few ancilla qubits may be added on top.

With this estimate, f7 needs about 22 qubits and f1 about 30, which is past what we can simulate. We kept f4 unchanged and reworked the rest:

Instance	Role (Annex A)	n	C	V	Solutions (M)	≈ Qubits (estimate)	Qubits (T2)

toy	tiny, checkable by hand (Part I example)	3	5	7	1	11	15
tiny	Annex A example	4	10	15	1	14	18
f4	medium, known optimum (Pisinger, unchanged); near the 20-qubit limit	4	11	23	1	15	20
f4_multi	several solutions	4	11	16	5	15	20
f4_none	no solution	4	11	24	0	15	20
f7_sub5	larger instance, oracle checked classically only	5	25	46	1	20	25

f7_sub5 keeps the first five items of f7 with the capacity to half (C = 25) and V set to the new optimum. The instances with several solutions and with none reuse f4 and only change V: below the optimum several subsets qualify, and above it (V = Opt + 1) none can. f1 is left out of the simulated tests and is mentioned as a limitation. All instances are written as problem DSL object models in `models/instances.py`; the original list stays in `instances/instances.md`.

After implementing T2 we had to revisit this table a second time. The estimate above counts only the item qubits and one register per constraint. The adder used in T2 [CDKM04] shares a single expression register between the two constraints, but it also needs a temporary register of the same width, a carry qubit, one flag qubit per constraint

and a phase qubit, so the real circuits are 4 to 5 qubits larger (column "Qubits (T2)"). f4 and its two variants land exactly on 20 qubits, so f4 now also plays the "near the 20-qubit limit" role. f7_sub5 grew to 25 qubits: we kept it, but only check its oracle classically on all 2^5 inputs and do not simulate it (see Limitations).

Classical baseline

We wrote `validation/baseline.py` to get the correct answer for every instance before running anything on Aer. It reads the same problem DSL models as T1 and runs two exact methods:

- **Brute force**, as the assignment suggests [TACS26, 7.3]. It tries all 2^n subsets and keeps the ones with weight ≤ C and value ≥ V. This gives us the full list of solutions, which we compare with what Grover measures, and their number M. With n ≤ 5 it checks at most 32 subsets.
- **Dynamic programming**, the usual exact method for knapsack [KPP04, ch. 2]. It finds the best value within capacity C in O(n·C), and the instance is feasible iff that value is ≥ V. It doesn't list the solutions, so we use it as a second check: the script confirms that both methods agree on every instance.

We left out greedy and other heuristics because they can miss the optimum.

We also use M to set the number of Grover rounds, `floor(π/4 · √(2^n / M))` [BBHT98]: 3 rounds for f4 (M = 1, 96% success), 1 round for f4_multi (M = 5), and nothing to amplify for f4_none (M = 0). Run it with `python -m validation.baseline`.

Transformations

Both transformations are plain Python functions over BESSER models, one function per rule, and each rule cites its reference in the code comment next to it (`transformations/t1.py`, `transformations/`

`t2.py`). The full concept mapping is in `docs/concept_list.md`.

T1: problem model → solution model (`transformations/t1.py`). The input is a `KnapsackProblem` with its items and the solver configuration model; the output is a `ConstraintSystem` of linear integer constraints, plus a trace `{created object: source object}`. The backward mapping (`transformations/backward.py`) uses it: a measured bitstring is decoded to unknowns with the T2 layout, and the T1 trace maps each unknown to its problem-model `Item`, so solutions are mapped back through the trace and checked on the original problem.

Rule	Problem DSL	Solution DSL	Reference
R1	`KnapsackProblem`	`ConstraintSystem`	Annex A, class C
R2	`Item`	`Unknown` y_i of 1 bit (1 = packed)	[GJ79] MP9
R3	capacity C	`Atom` Σ w_i·y_i – C ≤ 0	[KPP04]; Annex A
R4	minimum value V	`Atom` V – Σ v_i·y_i ≤ 0	Annex A (variants)
R5	feasibility question	root `And` of the two atoms	[GJ79] MP9
R6	solver configuration	`SolverConfig`, copied from the input and never defaulted	assignment §2

T2: solution model → quantum model (`transformations/t2.py`). The output is a BESSER `QuantumCircuit` that implements Grover search with a compute–mark–uncompute oracle.

Rule	Solution DSL	Quantum model	Reference
R1	`ConstraintSystem`	`QuantumCircuit` with one register and n classical bits	Annex A

R2	`Unknown`	search qubits, put in superposition with H	[Gro96]
R3	`LinearExpression` a0 + Σ a_i·y_i	expression register of B bits, two's complement, a0 loaded with X	[NC10]; Annex A
R4	`Term` a_i·y_i	addition of a_i controlled by y_i, with a ripple-carry adder	[CDKM04]; [VBE96]; [Dra00]
R5	`Atom` (≤ or =)	comparator that sets one flag qubit per atom	[NC10]; Annex A
R6	root `And`	multi-controlled X from all flags onto the phase qubit in |–⟩	[BBC+95]; [CEMM98]
R7	oracle	compute, mark, then run the compute blocks backwards	[Ben73]
R8	search register	diffuser H X (multi-controlled Z) X H	[Gro96]
R9	`SolverConfig.iterations`	number of rounds (zero allowed); shots go to T3	[Gro96]; [BBHT98]
R10	`Unknown` names	measurement of the search qubits, and decoding of the counts back to item names	traceability choice

Three design decisions are worth stating. First, the solver choices (rounds, shots, adder, register width, encoding) are read from the

configuration model, so neither transformation hard-codes them. Second, the oracle uses only primitive BESSER gates (X, H, Z with controls): BESSER's Qiskit generator turns `ComparisonGate` into an empty placeholder and `ArithmeticGate` into a Draper adder without constants, which would give a circuit that runs but is wrong, so we build the adder and the comparator ourselves. Third, the decoding is derived from the T2 qubit layout and not written by hand for each instance. T3 is not ours: it is BESSER's `QiskitGenerator`, and its output is never edited.

Validation results

`python -m demo` runs the whole chain on every instance: baseline → T1 (with the instance's solver configuration from `models/solver_configs.py`) → T2 → BESSER generator → Aer (1024 shots). It then decodes the bitstrings back into items using the T2 layout, compares them with brute force and writes `out/validation.md`. As suggested for class C [TACS26, 7.3], we measure the probability of getting a valid solution against the theory $\sin^2((2R+1)\theta)$, with $\theta = \arcsin\sqrt{M/N}$ [BBHT98], and count the oracle calls R.

Instance	Qubits	M	Rounds = oracle calls	P theory	P measured	Most frequent answer
toy	15	1	2	94.5%	95.5%	{a, b}
tiny	18	1	3	96.1%	95.5%	{a, b, c}
f4	20	1	3	96.1%	95.4%	{i2, i4}
f4_multi	20	5	1	95.7%	96.1%	{i2, i4} (one of the 5)
f4_none	20	0	0	0%	0%	none exists
f7_sub5	25	1	4	99.9%	not simulated	–

Every simulated instance lands within about 1 point of the theory (with 1024 shots one standard deviation is about 0.7 points, and Aer isn't seeded, so the numbers move a bit between runs), and the most frequent answer is always a real solution. f4_none behaves as it should: 0 rounds, nothing valid measured, answer "infeasible".

Building the validation also caught bugs we wouldn't have seen otherwise:

1. T1 had the Grover rounds fixed at 1, so f4 only reached about 47% instead of 96%. T2's self-check hid it because it computed its own rounds; the demo compares the rounds in the solution model with the optimum, and that's how we spotted it. Fix: the solver configuration is now an explicit model that T1 requires, with no hidden defaults.
2. T2 read 0 rounds as "missing" and fell back to 3, so f4_none ran 3 rounds. We noticed because its circuit was bigger than expected. T2 now accepts 0.
3. Qiskit orders bits little-endian, which is easy to get backwards. The decoded answers matching brute force on every instance confirm the mapping is right.

T2 also checks the oracle classically on all 2^n inputs, making sure it marks exactly the solutions and leaves every ancilla at 0.

Limitations

- **Grover rounds come from the baseline.** The number of rounds is an input of the chain, stored in the solver configuration model (`models/solver_configs.py`), and neither T1 nor T2 computes it. We set it to floor(π/4 · √(2^n / M)) [BBHT98], where M, the number of solutions, comes from the classical brute-force baseline; `validation/test_solver_config.py` checks that every configured value equals that formula. The consequence is that M has to be known before the quantum run, which is only possible for instances small enough to brute-force, so the quantum run validates the chain and does not speed anything up. For an unknown M, [BBHT98] describes a procedure that tries

randomly chosen round counts; we did not implement it.
- **Qubits.** The ripple-carry adder [CDKM04] adds 4 to 5 qubits over our estimate, so f7_sub5 (25 qubits) is only checked classically and f1 is out of reach. The QFT adder [Dra00] is in the metamodel but not implemented in T2.
- **Output.** The generated script prints raw counts. Turning them into items is done by the demo, because the BESSER template has no hook for it and generated code must not be edited.
- **Formulas.** T2 only handles an And of `<=/=` constraints, no Or.
- **No speedup.** At this size dynamic programming answers instantly, and simulating the circuit costs 2^qubits memory. The goal is a correct chain, not performance.
- **Setup.** The pip package of the BESSER fork is missing the Qiskit template, so `requirements.txt` installs BESSER in editable mode.

References

- [CDKM04] S. A. Cuccaro, T. G. Draper, S. A. Kutin, D. P. Moulton. "A new quantum ripple-carry addition circuit", arXiv:quant-ph/0410184, 2004.
- [Dra00] T. G. Draper. "Addition on a quantum computer", arXiv:quant-ph/0008033, 2000.
- [GJ79] M. R. Garey, D. S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, 1979 (problem MP9, Knapsack).
- [Gro96] L. Grover. "A fast quantum mechanical algorithm for database search", STOC, 1996.
- [VBE96] V. Vedral, A. Barenco, A. Ekert. "Quantum networks for elementary arithmetic operations", Physical Review A, 1996.
- [BBC+95] A. Barenco et al. "Elementary gates for quantum computation", Physical Review A, 1995.
- [CEMM98] R. Cleve, A. Ekert, C. Macchiavello, M. Mosca. "Quantum algorithms revisited", Proceedings of the Royal Society A, 1998.
- [Ben73] C. H. Bennett. "Logical reversibility of computation", IBM Journal of Research and Development, 1973.

- [NC10] M. A. Nielsen, I. L. Chuang. Quantum Computation and Quantum Information, 10th anniversary ed. Cambridge University Press, 2010.
- [KPP04] H. Kellerer, U. Pferschy, D. Pisinger. Knapsack Problems. Springer, 2004.
- [BBHT98] M. Boyer, G. Brassard, P. Høyer, A. Tapp. “Tight bounds on quantum searching”, Fortschritte der Physik, 1998.
- [Pis] D. Pisinger, low-dimensional 0/1 knapsack instances, as collected in https://github.com/dnlfm/knapsack-01-instances/tree/main/pisinger_instances_01_KP/low-dimensional
- [TACS26] Assignment: "TACS 2627 Assignment, QMDE"